

Milestone 3 Progress Summary

Project Setup and Development Environment, Economic Development in Latin America (App Phase)

1. Create GitHub Repository

Created the project's GitHub repository and established it as the single source of truth for the application code, dataset, documentation and configuration files.

- Repository: economic-development-latin-america (existing analysis repo, now extended for the app phase). Link:
<https://github.com/ManarMohammed78/economic-development-latin-america>
- The same repository is used for the app phase, as decided in Milestone 2. This keeps the cleaned dataset, documentation and new app code together on the main branch.
- Streamlit Community Cloud will deploy directly from this repo. No separate hosting repo is needed. Reusing the existing repo keeps the full history connected.

2. Create Project README

Created a new README for the app phase at README.md in the repository root. The file replaces the analysis only README.

- Project overview: what the app does, who it is for, to explore 12 indicators across 6 countries from 2014 to 2024, descriptive only
- MVP scope: Must have, Nice to have and Postponed, exactly as defined in Milestone 2
- Technology stack: Streamlit for frontend and backend, pandas, Plotly and Streamlit Community Cloud, with a short note on why this stack was chosen. The dataset is small and static so no database is needed for the first version
- Setup instructions: clone, create virtual environment, pip install -r requirements.txt, validate dataset, streamlit run app.py
- Data source: World Bank Open Data, list of 12 indicators and notes on the two known data gaps
- Project structure: folder tree showing app.py, data, pages, utils, .streamlit, scripts and docs
- Instructions for running the application locally: step by step commands, no API keys needed

3. Create Project Structure

Set up the initial folder and file structure according to the architecture defined in Milestone 2. The architecture is a single Streamlit app, CSV loaded with pandas, no database or separate API.

- Six pages were created to match the six low fidelity wireframes: Overview, Social Development, Investment and Technology, Insights, Data Table and About and Methodology

- Simple data flow as agreed in Milestone 2: cleaned CSV, then pandas with caching, then filtered in memory, then rendered by Streamlit. This keeps the flow clear and linked to the previous milestones
- All folders were created and placeholder files were added so the app runs immediately with `streamlit run app.py`

4. Set Up Development Environment

Configured the Python environment and installed the required dependencies, including Streamlit, pandas and Plotly.

- Created `requirements.txt` with streamlit 1.39.0, pandas 2.2.3, plotly 5.24.1 and numpy 1.26.4
- Verified locally with a virtual environment. The app starts on `http://localhost:8501` with no errors
- `utils/data_loader.py` uses `st.cache_data` so the CSV is loaded once per session. This matches the performance note from Milestone 2

5. Add and Validate Dataset

Added the cleaned dataset to the repository according to the architecture defined in Milestone 2.

- Location: `data/Latin_America_Economic_Development_Clean.csv`. The same cleaned file from the analysis phase was used. No re cleaning was needed
- Stored inside the repo so the app can load it with `pandas.read_csv` at runtime
- Verified that the application can successfully load the dataset using pandas and that the expected 792 observations and 12 indicators are available. Run: `python scripts/validate_dataset.py`. Result: 792 rows, 12 indicators, 6 countries, year range 2014 to 2024. All core checks passed. Argentina inflation missing before 2018 and Brazil poverty missing for all years, handled as documented.
- Missing data is handled exactly as in Milestone 1. It is never shown as zero. It is shown as No data available with a note.

6. Create GitHub Issues

Created GitHub Issues for the main development tasks required for Milestone 4, based on the MVP features and wireframes defined in Milestone 2.

- Drafted in `docs/github-issues-milestone-4.md`, ready to copy into GitHub. There are 9 issues, each specific enough to track progress
- Issue 1: Overview Page with headline KPIs and trend charts for GDP, unemployment and inflation
- Issue 2: Social Development Page with life expectancy, poverty where Brazil is excluded, and school enrollment

- Issue 3: Investment, Technology and Demographics Page with FDI, internet access and population
- Issue 4: Global Filters and Missing Data Handling for country, year range and indicator selector
- Issue 5: Insights Panel to browse the 16 documented insights by theme, nice to have
- Issue 6: Data Table View with sortable table synced with filters
- Issue 7: CSV Export to download filtered rows only
- Issue 8: About and Methodology Page with data source and link to White Paper
- Issue 9: Navigation and Layout Polish to keep filters persistent across pages
- Each issue includes labels, description, tasks and acceptance criteria. This matches the MVP must haves and the documented nice to haves

7. Define Branching Workflow

Defined and documented a simple Git workflow for development, including how branches, commits and merges will be handled.

- Documented in docs/branching-workflow.md
- main is always deployable. Streamlit Cloud deploys from it. No direct commits to main except small documentation fixes
- Work happens on short lived feature branches: feature/overview-page, feature/social-development-page, feature/filters-and-missing-data, and fix branches for bugs
- Commits are small and descriptive. Examples: feat: add Overview page, fix: handle missing Brazil poverty data, docs: update README
- Every feature is pushed and a pull request is opened from the feature branch to main. The PR links the related Issue, for example Closes 1, is reviewed, then squash merged. The feature branch is deleted after merge
- No develop branch is needed at this size. It can be added later if the team grows
- This workflow keeps GitHub as the single source of truth and matches the hosting choice where Streamlit Cloud auto deploys from main